Week 5, Day 3: Lambda, ECS and EKS, Load Balancers, CloudWatch and Multi-Source BFS

Prepared for Upendra Kumar Sharma | 4 hours, plus an optional 90-minute lab | Main topic: choosing and running compute on AWS, load balancing and monitoring | DSA: Rotting Oranges and Pacific Atlantic Water Flow

Today's Schedule

- **0:00 to 0:20** Compare the compute options (section 1).
- **0:20 to 1:05** Study Lambda (section 2).
- **1:05 to 1:50** Study ECR, ECS, Fargate and EKS (section 3).
- **1:50 to 2:25** Study load balancers (section 4).
- **2:25 to 2:50** Study CloudWatch and monitoring (section 5), then the deployment decision for your project (section 6).
- **2:50 to 3:40** DSA: solve the 2 problems in section 8. Allow 20 minutes for the first and 30 for the second.
- **3:40 to 4:10** Answer the interview questions aloud (section 9) and add 10 flashcards.
- **Optional (90 min):** the hands-on lab in section 7. Remember to delete the load balancer and the ECS service afterwards, because they cost money every hour.

1. Choosing Compute on AWS

- **EC2 (virtual machines):** full control of the operating system, long-running or special workloads, but you patch, scale and secure the servers.
- **Containers on ECS or EKS:** package the app once and run it anywhere on the platform. You manage images and service settings, and AWS manages scheduling. **Fargate** removes the servers too.
- **Lambda (functions):** run code in response to events, pay only while it runs, no servers to manage, scales to zero.
- **App Runner:** the simplest option. Point it at an image or repository and get a managed HTTPS service.
- **Elastic Beanstalk:** an older platform-as-a-service layer over EC2, still found in existing systems.

How to choose (a framework to say aloud)

- **Spiky, infrequent or event-driven work, glue code, short tasks:** Lambda.

- **Steady web services and APIs, many services:** containers on ECS Fargate (simpler) or EKS (when you need Kubernetes).
- **Special hardware, licensing, or full OS control:** EC2.
- **Cost:** at steady high load, always-on containers are usually cheaper than Lambda. At low or spiky load, Lambda wins because you pay nothing when idle.

2. AWS Lambda

The model: you upload a function and configure a **trigger**. AWS runs it in an **execution environment** when events arrive, and scales the number of environments automatically.

Lifecycle and cold starts

- The first request in a new environment pays an **init phase** (start the runtime, load your code, run initialization code). That is a **cold start**. Later requests reuse the **warm** environment and skip it.
- Initialize expensive things (SDK clients, connections) **outside the handler**, so warm invocations reuse them.
- **Java cold starts are the slow ones**, because of JVM start-up, class loading and, with Spring, context start-up. Mitigations:
  - **SnapStart** (for supported Java versions) restores a snapshot of the initialized environment, which cuts start-up dramatically.
  - **Provisioned concurrency** keeps a number of environments initialized (it costs money).
  - A **lighter framework** (plain handlers, Micronaut, Quarkus, Spring Cloud Function with care), or a **GraalVM native image**.
  - **More memory**, because CPU scales with memory, so init runs faster. Also **ARM (Graviton)** functions are cheaper.
  - Avoid heavy classpath scanning and unused dependencies.

Limits to know (check current values)

- Maximum run time **15 minutes**. Memory from **128 MB to 10 GB**, with CPU proportional to memory. Temporary disk (/tmp) of 512 MB by default, configurable higher. Deployment package limits (about 50 MB zipped, 250 MB unzipped), or a **container image up to 10 GB**.
- **Concurrency:** a regional account limit (1,000 by default, raisable). **Reserved concurrency** guarantees and also caps a function. Provisioned concurrency keeps warm capacity.

Invocation types and triggers

- **Synchronous:** the caller waits for the result. API Gateway, an Application Load Balancer, a function URL.
- **Asynchronous:** the event is queued and the caller does not wait. S3, SNS, EventBridge. Lambda **retries failures twice by default**, and you can set an on-failure destination or a DLQ.
- **Polling (event source mapping):** Lambda polls a source and invokes your function with **batches**. SQS, Kinesis, DynamoDB Streams.

Lambda with SQS (a very common pattern)

- Lambda reads messages in batches and **deletes them if the function succeeds**. If the function throws, the **whole batch** returns to the queue, so successful messages are processed again.
- Turn on **partial batch responses**: the function returns the ids of only the failed messages, and only those are retried.
- Set the queue's **visibility timeout to at least six times the function timeout** (AWS guidance), and configure a **DLQ** with maxReceiveCount.
- Handlers still must be **idempotent**, because delivery is at-least-once.

```
public class OrderEventHandler implements RequestHandler<SQSEvent,
SQSBatchResponse> {

    private final OrderProcessor processor = new OrderProcessor();   //
created once, reused when warm

    @Override
    public SQSBatchResponse handleRequest(SQSEvent event, Context context) {
        List<SQSBatchResponse.BatchItemFailure> failures = new ArrayList<>();
        for (SQSEvent.SQSMessage message : event.getRecords()) {
            try {
                processor.process(message.getBody());
            } catch (Exception ex) {
                failures.add(new
SQSBatchResponse.BatchItemFailure(message.getMessageId()));
            }
        }
        return new SQSBatchResponse(failures);
    }
}
```

Cost: you pay per request and per GB-second of run time, and there is a monthly free allowance. It is very cheap at low volume, and can be expensive for steady heavy traffic.

Good uses: processing S3 uploads (thumbnails, scanning, parsing), scheduled jobs (EventBridge Scheduler), webhooks, glue between AWS services, lightweight APIs.

Poor uses and pitfalls: long-running jobs, steady high-throughput services, latency-critical Java without SnapStart or provisioned concurrency, and **database connections**. Many concurrent functions can open too many connections to RDS, so use **RDS Proxy**.

Observability: logs go to CloudWatch automatically. Watch the **Errors, Throttles, Duration and ConcurrentExecutions** metrics, and the REPORT line in the logs, which shows the duration and **init duration** of each invocation.

3. Containers on AWS: ECR, ECS, Fargate and EKS

ECR (Elastic Container Registry): a private registry for your images. It scans images for known vulnerabilities, supports lifecycle policies to remove old images, and integrates with IAM. Your pipeline builds the image and pushes it here.

ECS (Elastic Container Service) concepts

- **Cluster:** a logical grouping of capacity.
- **Task definition:** the blueprint for a task. It names the image, CPU and memory, ports, environment variables, **secrets** (pulled from Secrets Manager or Parameter Store), the log configuration (awslogs driver to CloudWatch), and the **two IAM roles** below.
- **Task:** one running instance of a task definition (one or more containers that start and stop together).
- **Service:** keeps a **desired count** of tasks running, replaces failed ones, performs **rolling deployments**, registers tasks with a load balancer target group, and can **auto scale**.
- **Networking (awsvpc mode):** each task gets **its own network interface, private IP and security group**. Place tasks in private subnets, and allow traffic only from the load balancer's security group.

The two roles (a favorite interview question)

- **Task execution role:** used by **ECS itself** to pull the image from ECR, write logs to CloudWatch and fetch secrets for injection.
- **Task role:** used by **your application code** to call AWS APIs, such as reading S3 or sending to SQS. Give it least privilege.

Launch types

- **Fargate:** serverless containers. You choose CPU and memory per task, and AWS runs it. No instances to patch or scale. You pay per vCPU and GB per second. Simplest to operate.

- **EC2:** you run and scale a fleet of instances that host your tasks. More control, and often cheaper at large steady scale, but more work.

Deployments: a **rolling update** replaces tasks gradually, controlled by minimum healthy percent and maximum percent. Enable the **deployment circuit breaker** to roll back automatically if new tasks keep failing. **Blue/green** deployments through CodeDeploy shift traffic between two task sets.

Service discovery between services: ECS Service Connect or AWS Cloud Map give tasks stable service names (similar to Compose DNS names), or you put an internal load balancer in front.

EKS (Elastic Kubernetes Service)

- AWS runs the **Kubernetes control plane** (you pay an hourly fee per cluster). You run worker capacity as **managed node groups** (EC2), on **Fargate**, or with **Karpenter** for fast autoscaling.
- **IAM for pods:** IRSA or EKS Pod Identity gives each Kubernetes service account its own IAM role, the equivalent of the ECS task role.
- The **AWS Load Balancer Controller** creates ALBs and NLBs from Kubernetes Ingress and Service objects.
- **Choose EKS** when you need Kubernetes (portability, its ecosystem, existing team skills, advanced scheduling). **Choose ECS** when you want the simplest way to run containers on AWS. Kubernetes itself is Day 5.

4. Elastic Load Balancing

The types

- **Application Load Balancer (ALB), layer 7:** understands HTTP, HTTPS, gRPC and WebSockets. **Routing rules** by path, host name, header, query string or method send requests to different **target groups**. Targets can be instances, IP addresses (containers) or Lambda functions. It supports redirects, fixed responses, sticky sessions with cookies, OIDC and Cognito authentication, and AWS WAF. **The default choice for web applications and APIs.**
- **Network Load Balancer (NLB), layer 4:** TCP, UDP and TLS, extremely high throughput and low latency, **static IP addresses** per AZ, and it preserves the client source IP. Use it for non-HTTP protocols, for static IPs, or in front of PrivateLink services.
- **Gateway Load Balancer** is for network appliances such as firewalls. **Classic Load Balancer** is legacy.

Concepts

- **Listener:** a protocol and port (for example 443) with rules. **Target group:** the set of targets with a **health check**.
- **Health checks:** path, interval, healthy and unhealthy thresholds, and expected status codes. Point them at a **readiness endpoint** (for example /actuator/health/readiness), not just any page. A target that fails is taken out of rotation.
- **TLS termination:** the load balancer holds the certificate (from **AWS Certificate Manager**, free for public certificates), and traffic to targets may be plain HTTP inside the VPC or re-encrypted. Add an HTTP to HTTPS redirect.
- **Cross-zone load balancing** spreads requests evenly across targets in all AZs. Deploy at least **two AZs**.
- **Deregistration delay (connection draining):** when a target is removed, in-flight requests get time to finish (300 seconds by default). Align it with your application's **graceful shutdown**.
- **Idle timeout (60 seconds by default):** if your application closes idle connections **before** the load balancer does, the load balancer may send a request on a connection that just closed, causing sporadic **502** errors. The fix is to set the application's keep-alive timeout **longer than the load balancer's idle timeout**.
- **Auto scaling policies** can scale on **ALB request count per target** or target response time.
- **Access logs** to S3 help with debugging and auditing.

Reading the errors

- **502 Bad Gateway:** the target closed the connection or sent an invalid response.
- **503 Service Unavailable:** there are **no healthy targets**.
- **504 Gateway Timeout:** the target did not answer within the timeout.

ALB vs API Gateway: API Gateway is a managed API front door with throttling, API keys and usage plans, request validation, built-in authorizers and direct Lambda integration, and a per-request price. An **ALB** is cheaper for steady high traffic to containers and instances. Your Spring Cloud Gateway can sit behind an ALB, and **Route 53** (an alias record) and **CloudFront** can sit in front.

5. CloudWatch and Monitoring

- **Metrics:** time series organized by **namespace** and **dimensions** (for example AWS/ApplicationELB, with a load balancer dimension). Standard resolution is one minute, with high-resolution options. EC2 basic monitoring is every 5 minutes, and detailed monitoring every minute (extra cost). Memory and disk usage on EC2 are **not** collected by default, and need the CloudWatch agent.

- **Custom metrics:** publish with the API, with the **Embedded Metric Format** in logs, or from Micrometer. Keep dimensions low in cardinality, because every dimension combination is a billed metric.
- **Logs: log groups** hold **log streams**. **Set a retention period**, since the default never expires and log storage and ingestion cost real money. **Logs Insights** queries search logs, **metric filters** turn log patterns into metrics, and **subscription filters** stream logs elsewhere.
- **Alarms:** watch a metric against a threshold (or an anomaly detection band) and move between **OK, ALARM and INSUFFICIENT_DATA**. Actions: notify through **SNS**, trigger Auto Scaling, or act on an EC2 instance. **Composite alarms** combine several alarms to cut noise.
- **Dashboards** for a shared view. **X-Ray** or the OpenTelemetry collector (ADOT) for distributed tracing. **Container Insights** for ECS and EKS metrics.
- **Related services, not to be confused: CloudTrail** records API calls (who did what, for audit). **AWS Config** records resource configuration and compliance over time. **CloudWatch** is for performance, logs and alarms.

For a Spring Boot service: write **JSON logs to standard output** (the awslogs driver or Fluent Bit ships them), include the trace id, expose Micrometer metrics, and use the readiness endpoint for load balancer health checks.

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

Alarms worth having: ALB 5xx rate, target response time p95, unhealthy host count, ECS CPU and memory, RDS CPU, connections, free storage and replica lag, SQS age of the oldest message and DLQ depth, Lambda errors and throttles, and a billing alarm.

6. Deciding How to Deploy Your Project (Preview of Day 6)

A reasonable plan for the capstone, with the reasoning you can quote

- **Images in ECR**, built and pushed by your CI pipeline.
- **ECS on Fargate** for the five services, because it needs no server management and teaches the deployment pipeline end to end. Place them in the **private application subnets**, with the security group chain from Day 1.
- **An ALB** in the public subnets in front of the gateway, with an ACM certificate if you have a domain.
- **RDS PostgreSQL** in the data subnets, and **ElastiCache Redis**.

- **Messaging:** SNS and SQS (the mapping from Day 2) to avoid the cost of managed Kafka, or Kafka on a small instance for learning only. Document what production would use.
- **Secrets** in Secrets Manager, injected through the task definition. **Logs and alarms** in CloudWatch.
- **Cost:** the ALB, Fargate tasks, RDS and NAT all bill by the hour. Create the stack, test it, take screenshots and notes, then **tear it down**. Infrastructure that you can recreate from a script is better than infrastructure left running.
- **EKS** comes after Day 5, as a stretch goal or a talking point rather than a requirement.

7. Optional Hands-On Lab (90 Minutes)

1. **ECR.** Create a repository, log in with the CLI, then build, tag and push your gateway or order-service image.
2. **ECS Fargate.** Create a cluster, a task definition (with both roles and the awslogs configuration), and a service with 2 tasks behind a new ALB. For the lab it is fine to use public subnets with a public IP. Use the readiness endpoint for health checks.
3. **Break it on purpose.** Make the health endpoint fail and watch tasks being marked unhealthy and replaced. Deploy a broken image and watch the deployment circuit breaker roll back.
4. **ALB rules.** Add a path-based rule to a second target, and see the 503 you get when a target group has no healthy targets.
5. **Lambda.** Create a function triggered by an S3 upload that logs the object key and size (use Python if you are short of time, or Java). Read the REPORT lines and compare the cold and warm durations. If you use Java, enable SnapStart and compare again.
6. **CloudWatch.** Run a Logs Insights query over your service logs, create an alarm on the ALB 5xx count that emails you through SNS, and build a small dashboard.
7. **Clean up:** delete the ECS service and cluster, the ALB and target groups, ECR images, the Lambda, and set log retention. Check billing tomorrow.

8. DSA: Multi-Source BFS and Reverse Search (2 Problems)

Problem 1: Rotting Oranges

- **The problem:** a grid where 0 is empty, 1 is a fresh orange and 2 is a rotten one. Each minute, every fresh orange next to a rotten one (up, down, left, right) becomes rotten. Return the minutes until no fresh orange remains, or -1 if that is impossible.

- **Idea: multi-source BFS.** Put **all** rotten oranges into the queue at the start, and count the fresh ones. Process the queue **one level at a time**. Each level is one minute. Rot the fresh neighbors, decrease the fresh count and enqueue them. If the fresh count reaches 0, return the minutes. If the queue empties while fresh oranges remain, return -1.
- **Why BFS:** BFS spreads outward at the same speed from every source, which is exactly how the rot spreads.
- **Complexity:** O(R x C) time and space.
- **Edge cases:** no fresh oranges (0 minutes), no rotten oranges but some fresh (-1), a fresh orange cut off by empty cells (-1), an empty grid.

```
public int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length, fresh = 0;
    Queue<int[]> queue = new ArrayDeque<>();
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) queue.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;
        }
    }

    int minutes = 0;
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    while (!queue.isEmpty() && fresh > 0) {
        int size = queue.size();                    // everything in the
queue rots at this minute
        for (int i = 0; i < size; i++) {
            int[] cell = queue.poll();
            for (int[] d : dirs) {
                int nr = cell[0] + d[0], nc = cell[1] + d[1];
                if (nr >= 0 && nc >= 0 && nr < rows && nc < cols && grid[nr]
[nc] == 1) {
                    grid[nr][nc] = 2;
                    fresh--;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}
```

Problem 2: Pacific Atlantic Water Flow

- **The problem:** a grid of heights. Water flows from a cell to a neighbor of **equal or lower** height. The Pacific Ocean touches the top and left edges, and the Atlantic touches the bottom and right edges. Return all cells from which water can reach **both** oceans.
- **Idea: search in reverse.** Checking every cell by flowing downhill repeats a lot of work. Instead, start from the **ocean borders** and move **uphill** (to neighbors of equal or greater height). Do this once from the Pacific border and once from the Atlantic border, marking reachable cells in two boolean grids. The answer is every cell marked in **both**.
- **Complexity:** O(R x C) time and space, since each cell is visited at most twice.
- **Edge cases:** a single cell (it touches both oceans), a completely flat grid (every cell qualifies), a single row or column.
- **Note:** the recursive DFS can go as deep as the number of cells, so for large grids use an iterative BFS with a queue.

```
public List<List<Integer>> pacificAtlantic(int[][] heights) {
    int rows = heights.length, cols = heights[0].length;
    boolean[][] pacific = new boolean[rows][cols];
    boolean[][] atlantic = new boolean[rows][cols];

    for (int r = 0; r < rows; r++) {
        dfs(heights, pacific, r, 0);              // left edge touches the
Pacific
        dfs(heights, atlantic, r, cols - 1);      // right edge touches the
Atlantic
    }
    for (int c = 0; c < cols; c++) {
        dfs(heights, pacific, 0, c);              // top edge
        dfs(heights, atlantic, rows - 1, c);      // bottom edge
    }

    List<List<Integer>> result = new ArrayList<>();
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (pacific[r][c] && atlantic[r][c]) result.add(List.of(r, c));
        }
    }
    return result;
}

private void dfs(int[][] h, boolean[][] seen, int r, int c) {
    seen[r][c] = true;
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
```

```
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        if (nr >= 0 && nc >= 0 && nr < h.length && nc < h[0].length
                && !seen[nr][nc] && h[nr][nc] >= h[r][c]) {
            dfs(h, seen, nr, nc);
        }
    }
}
```

Patterns to remember: a problem with several starting points that spread at the same speed is a **multi-source BFS**, and counting levels gives time. When a forward search from every cell is too slow, **search backward from the targets**.

9. Interview Questions to Answer Aloud

1. How would you choose between EC2, ECS on Fargate, EKS and Lambda for a new service?
2. What is a Lambda cold start? Why is it worse for Java, and how do you reduce it?
3. What are Lambda's invocation types, and how do retries differ between synchronous and asynchronous invocation?
4. How does Lambda process an SQS queue? What are partial batch responses, and how should the visibility timeout be set?
5. How do Lambda and RDS interact badly, and what fixes it?
6. Explain ECS tasks, task definitions and services. What is the difference between the task role and the task execution role?
7. Fargate vs the EC2 launch type: trade-offs.
8. When would you choose EKS over ECS?
9. ALB vs NLB vs API Gateway: when do you use each?
10. How do target group health checks work, and what should a health endpoint check?
11. What causes 502, 503 and 504 errors from an ALB?
12. How do deregistration delay and the idle timeout relate to graceful shutdown and keep-alive settings?
13. Which CloudWatch alarms would you set for a containerized API with a database?
14. CloudWatch vs CloudTrail vs AWS Config: what does each answer?

10. End-of-Day Checklist

- [ ] I can choose between EC2, containers and Lambda and defend the choice with cost and operations arguments.

- [ ] I can explain Lambda cold starts, SnapStart, SQS integration and partial batch failures.
- [ ] I can explain ECR, ECS tasks, services, the two IAM roles, and Fargate vs EC2.
- [ ] I can explain ALB listeners, target groups, health checks, draining and the 502, 503 and 504 errors.
- [ ] I can list the CloudWatch features and the alarms a production service needs.
- [ ] I solved Rotting Oranges and Pacific Atlantic and can explain multi-source BFS and searching backward.
- [ ] (Optional) I deployed a service on Fargate behind an ALB, broke it on purpose, and deleted everything afterwards.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 5, Day 4): Docker multi-stage builds, networking and image hygiene, CI/CD with Jenkins and GitHub Actions, plus union find problems.